DAY 11 — ANSWERS

React Components, State & Forms

Objective

Build a React student registration form that accepts a student name, email, and course; validates the fields; and displays submitted details. The solution uses functional components, useState, event handling, controlled inputs, and conditional rendering.

1. Key Concepts

Concept	Explanation
Functional component	A JavaScript function that returns JSX for the user interface.
Props	Values passed from a parent component to a child component. This single-component solution does not need props.
State	Data that can change while the application runs. Updating state causes React to update the UI.
useState	A React Hook used to create and update state in a functional component.
Event handling	Functions such as onChange and onSubmit respond to user actions.
Controlled component	An input whose value is controlled by React state.
Conditional rendering	Displaying content only when a condition is true, such as showing details after successful submission.
Form validation	Checking required fields and confirming that the email has a basic valid format.

2. Project Setup

If you do not already have a React project, run these commands in the terminal:

```
npm create vite@latest day11-student-form -- --template react
cd day11-student-form
npm install
npm run dev
```

Replace src/App.jsx and src/App.css with the following code.

3. src/App.jsx — Complete Code

```
import { useState } from "react";
import "./App.css";

function StudentRegistrationForm() {
const [formData, setFormData] = useState({
name: "",
email: "",
course: "",
});
const [errors, setErrors] = useState({});
const [submittedStudent, setSubmittedStudent] = useState(null);

function handleChange(event) {
const { name, value } = event.target;
setFormData((previousData) => ({
...previousData,
[name]: value,
}));
}

function validateForm() {
const newErrors = {};

if (!formData.name.trim()) {
newErrors.name = "Please enter the student's name.";
}

const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!formData.email.trim()) {
newErrors.email = "Please enter an email address.";
} else if (!emailPattern.test(formData.email)) {
newErrors.email = "Please enter a valid email address.";
```

```
}

if (!formData.course) {
newErrors.course = "Please select a course.";
}
return newErrors;
}

function handleSubmit(event) {
event.preventDefault();
const validationErrors = validateForm();
setErrors(validationErrors);

if (Object.keys(validationErrors).length === 0) {
setSubmittedStudent({ ...formData });
} else {
setSubmittedStudent(null);
}
}

function handleReset() {
setFormData({ name: "", email: "", course: "" });
setErrors({});
setSubmittedStudent(null);
}

return (
<main className="page">
<section className="form-card">
<p className="eyebrow">STUDENT PORTAL</p>
<h1>Student Registration</h1>
<p className="description">
Enter your details to register for a course.
</p>

<form onSubmit={handleSubmit} noValidate>
<label htmlFor="name">Student Name</label>
<input
id="name"
name="name"
type="text"
value={formData.name}
onChange={handleChange}
placeholder="Enter your full name"
aria-invalid={Boolean(errors.name)}
/>
{errors.name && <p className="error">{errors.name}</p>}

<label htmlFor="email">Email Address</label>
<input
id="email"
name="email"
type="text"
value={formData.email}
onChange={handleChange}
placeholder="name@example.com"
aria-invalid={Boolean(errors.email)}
/>
{errors.email && <p className="error">{errors.email}</p>}

<label htmlFor="course">Select Course</label>
<select
id="course"
name="course"
value={formData.course}
onChange={handleChange}
aria-invalid={Boolean(errors.course)}
>
<option value="">Choose a course</option>
<option value="React">React</option>
<option value="Python">Python</option>
<option value="JavaScript">JavaScript</option>
<option value="Web Development">Web Development</option>
</select>
{errors.course && <p className="error">{errors.course}</p>}

<div className="form-actions">
<button type="submit">Register Student</button>
<button type="button" className="reset-button" onClick={handleReset}>
Reset
</button>
</div>
</form>
</section>

{submittedStudent && (
<section className="result-card" aria-live="polite">
<h2>Registration Successful!</h2>
<p><strong>Student Name:</strong> {submittedStudent.name}</p>
<p><strong>Email:</strong> {submittedStudent.email}</p>
<p><strong>Course:</strong> {submittedStudent.course}</p>
</section>
)}
```

```
</main>
);
}

export default StudentRegistrationForm;
```

4. src/App.css — Complete Code

```
* {
box-sizing: border-box;
}

body {
margin: 0;
min-width: 320px;
min-height: 100vh;
font-family: "Segoe UI", Arial, sans-serif;
background: #f4f6fb;
color: #20243a;
}

button, input, select {
font: inherit;
}

.page {
width: min(92%, 760px);
margin: 42px auto;
}

.form-card, .result-card {
background: #ffffff;
padding: 30px;
border: 1px solid #e1e6f0;
border-radius: 16px;
box-shadow: 0 8px 28px rgba(28, 77, 148, 0.08);
}

.eyebrow {
margin: 0 0 8px;
color: #1c4d94;
font-size: 12px;
font-weight: 700;
letter-spacing: 1.5px;
}

h1, h2 {
color: #1c4d94;
}

h1 {
margin: 0 0 10px;
font-size: 30px;
}

.description {
margin: 0 0 25px;
color: #687187;
line-height: 1.6;
}

form {
display: grid;
gap: 10px;
}

label {
margin-top: 8px;
font-size: 14px;
font-weight: 600;
}

input, select {
width: 100%;
min-height: 46px;
padding: 11px 13px;
border: 1px solid #cfd6e4;
border-radius: 8px;
background: #ffffff;
color: #20243a;
outline: none;
}

input:focus, select:focus {
border-color: #1c4d94;
box-shadow: 0 0 0 3px rgba(28, 77, 148, 0.12);
}

[aria-invalid="true"] {
border-color: #c0394b;
}

.error {
margin: -3px 0 4px;
color: #c0394b;
font-size: 13px;
}

.form-actions {
```

```
display: flex;
flex-wrap: wrap;
gap: 12px;
margin-top: 18px;
}

.form-actions button {
min-height: 44px;
padding: 11px 18px;
border: 1px solid #1c4d94;
border-radius: 8px;
background: #1c4d94;
color: #ffffff;
font-weight: 600;
cursor: pointer;
}

.form-actions button:hover {
background: #153b73;
}

.form-actions .reset-button {
background: #ffffff;
color: #1c4d94;
}

.form-actions .reset-button:hover {
background: #edf3fc;
}

.result-card {
margin-top: 24px;
border-left: 5px solid #16845b;
}

.result-card h2 {
margin-top: 0;
color: #16845b;
font-size: 22px;
}

.result-card p {
line-height: 1.7;
overflow-wrap: anywhere;
}

@media (max-width: 600px) {
.page {
margin: 24px auto;
}

.form-card, .result-card {
padding: 22px 17px;
}

h1 {
font-size: 25px;
}

.form-actions {
flex-direction: column;
}

.form-actions button {
width: 100%;
}
}
```

5. How It Works

useState: Stores form values, validation errors, and submitted student details.

handleChange: Updates state when the user types or selects a course.

handleSubmit: Prevents default submission, validates the fields, and displays the details only if there are no errors.

Controlled inputs: Input values come from React state and are updated using onChange.

Conditional rendering: The result card appears only after a valid submission.

Validation: Name is required, email must be in a basic valid format, and a course must be selected.

Reset: Clears the form, validation messages, and displayed result.

6. Testing and Expected Output

1. Submit an empty form: required-field errors appear.

2. Enter an invalid email such as abc: an email error appears.

3. Enter a name, a valid email such as student@example.com, and select a course.

4. Click Register Student: a success card displays the submitted name, email, and course.

5. Click Reset: all fields and messages clear.

Note: This is a frontend practice project. Submitted data is stored in React state only and is not saved to a database.